

균형잡힌 수열 - 풀이

작성자: 장근영

부분문제 1

아래 성질을 관찰할 수 있다.

- 균형잡힌 수열의 길이는 $2^n - 1$ 꼴로 나타낼 수 있다.
- 길이 $2^n - 1$ 의 균형잡힌 수열의 길이 $2^m - 1$ 의 접두사/접미사는 균형잡힌 수열이다. ($m \leq n$)

높이 k 인 균형잡힌 수열 내의 균형잡힌 부분수열의 개수를 $f(k)$ 로 정의하자. 전체 수열의 중앙값은 유일한 최댓값이므로, 이 값이 부분수열의 중심이 아닌 위치에 포함되면 최댓값 조건이 성립하지 않는다. 그리고 균형잡힌 수열의 접두사는 균형잡힌 수열이라는 관찰에 의해 중앙값을 중심으로 하는 k 개의 부분수열은 균형잡힌 수열이다. 따라서 $f(k) = k + 2f(k-1)$ 을 만족한다. 점화식을 사용해 $O(\log N)$ 에 문제를 해결할 수 있다.

부분문제 2

각 원소의 크기가 3 이하인 경우에는 균형잡힌 수열 a 을 아래와 같은 경우로 분류할 수 있다.

- a 의 길이가 1
- $a_0 < a_1 > a_2$
- $a = [1, 2, 1, 3, 1, 2, 1]$

$O(N)$ 에 모든 경우를 셀 수 있다.

부분문제 3

원소를 변경하면서 생기거나 사라지는 균형잡힌 수열의 개수를 세주면 된다. 균형잡힌 수열의 길이는 최대 7이기 때문에 각 쿼리당 $O(1)$ 에 균형잡힌 수열 개수의 변화를 세줄 수 있다.

부분문제 4

길이가 $2^k - 1$ 형태인 모든 부분 구간 $A[i..j]$ 에 대하여 균형 잡힌 수열인지 여부를 판별하자. 판별 조건은 해당 구간의 중앙값이 구간 내 유일한 최댓값이어야 하며, 이를 기준으로 나뉜 좌우 부분 구간이 다시 재귀적으로 균형 잡힌 수열이어야 한다.

길이 $L = 2^k - 1$ 인 하나의 구간을 판별할 때, 최댓값 검증 및 재귀 호출을 수행하면 $O(L \log L)$ 의 연산이 필요하다. 이를 모든 구간에 대해 합산할 경우, 전체 시간 복잡도는 $O(N^2 \log N)$ 이 됩니다.

부분문제 5

쿼리의 개수가 적으므로, 변경 발생 시 전체 상태를 다시 계산하는 방식을 택한다. 이때 Dynamic Programming을 사용하여 작은 구간부터 상향식으로 계산하면 효율성을 높일 수 있다. 특히, 자식 구간이 균형 잡힌 수열임이 확인되었다면 자식 구간의 중앙값은 이미 해당 구간의 최댓값임이 보장된다. 따라서 전체를 순회할 필요 없이 현재 중앙값과 좌우 자식의 중앙값만을 비교하여 최댓값 조건을 $O(1)$ 에 검증할 수 있으며, 이를 통해 쿼리당 $O(N \log N)$ 에 해결 가능하다.

부분문제 6

고정된 k 에 대해 특정 원소를 포함하는 길이 $2^k - 1$ 인 균형잡힌 수열은 최대 2개까지만 존재할 수 있다. 만약 3개 이상 존재한다고 가정하면, 그중 인접한 두 수열의 시작점 차이는 $2^{k-1} - 1$ 이하가 된다. 이는 필연적으로 한 수열이 다른 수열의 중앙값(해당 구간의 유일한 최댓값)을 자신의 구간 내 비중앙 위치에 포함하게 됨을 의미하며, 이는 최댓값 조건에 위배되어 모순이 발생한다.

이 관찰에 따라, 특정 원소의 값이 변경되었을 때 영향을 받아 재검사가 필요한 구간의 수는 각 높이마다 상수 개로 제한된다. 따라서 높이가 낮은 구간부터 상향식으로 올라가며 변경된 부분만을 갱신하면, 각 단계에서의 연산을 $O(1)$ 에 수행할 수 있습니다. 결과적으로 각 쿼리를 트리의 높이에 비례하는 시간 복잡도인 $O(\log N)$ 에 처리하여 전체 문제를 효율적으로 해결할 수 있습니다.

통신망 2 - 풀이

작성자: 구재현

서브태스크 1

제한이 아주 작기 때문에 다양한 풀이가 있습니다. 모든 $0 \leq t \leq T$ 에 대해서, t 초의 통신망의 연결 컴포넌트를 DFS (Flood-Fill) 로 구합니다. x 와 y 가 t 초에 연결되어 있다는 것은 t 초의 통신망 상태에서 x 와 y 가 같은 컴포넌트에 있다는 것입니다. $comp[t][v]$ 를, t 초에 정점 v 가 있는 연결 컴포넌트의 번호라고 정의하면, 이는 $comp[t][x] = comp[t][y]$ 라는 것과 동치입니다. 모든 쿼리에 대해 y 를 고정하고, 위 조건을 모든 시간 $l, l+1, \dots, r$ 에 대해 확인할 경우 $O(TS + QNT)$ 시간에 문제를 해결할 수 있습니다.

서브태스크 2

서브태스크 1과 동일하게 진행하나, 쿼리 처리 부분을 최적화합니다. y 를 고정했을 경우, 우리가 알고 싶은 것은 모든 $l \leq i \leq r$ 에 대해 $comp[x][i] = comp[y][i]$ 인지의 여부입니다. 이는 두 문자열의 부분문자열이 일치하는지를 판별하는 문제와 동일합니다. $O(TN)$ 시간에 각 문자열의 해시를 전처리해두면, 각 y 에 대해 이를 $O(1)$ 시간에 판별할 수 있습니다. 고로 전체 문제를 $O(TS + TN + QN)$ 시간에 해결할 수 있습니다.

서브태스크 3

오프라인 동적 연결성 알고리즘을 사용하여 전체 문제를 $O(N + S \log S \log N + Q \log N)$ 시간에 해결할 수 있습니다. 이 알고리즘에 대해서는 여러 자료에서 잘 설명되어 있으나, 풀이의 완결성을 위해 이 글에서 다시 설명합니다.

입력으로 주어진 간선 갱신을, "시간 구간 $[l, r]$ 에 간선 (u, v) 추가"의 형태로 변환합니다. 시간 축에 대해 세그먼트 트리를 만들면, 위에서 설명한 간선 갱신은 $O(\log n)$ 개의 노드에 간선 (u, v) 을 추가하는 것으로 해석할 수 있습니다. 각 시간의 연결 상태는, 해당 시간 위치에서 루트로 가는 경로에 있는 모든 노드의 간선 합집합을 그래프로 두었을 때, 연결 컴포넌트의 상태와 동일합니다.

일반적인 Union Find를 약간 수정해서 다음과 같은 기능을 지원하게끔 구현할 수 있습니다.

- Worst-case $O(\log n)$ 시간에 두 집합 union.
- $O(\log n)$ 시간에 집합 size 계산.
- $O(\log n)$ 시간에 마지막으로 진행한 연산 Undo.

구체적으로, path compression을 하지 않고 대신 union-by-size로 시간 복잡도를 보장해 주면 됩니다.

위에서 구성한 세그먼트 트리를 top-down으로 순회합니다. 구체적으로, DFS 함수가 특정 노드를 방문하면, 해당 노드에 있는 모든 간선을 Union-find에 추가하고, 재귀적으로 자식을 방문한 후, 노드를 떠날 때 해당 노드에서 합쳐준 간선들을 모두 undo해줍니다. 이렇게 할 경우 다음 두 invariant가 보장됩니다:

- 특정 노드를 방문하고 간선을 다 추가한 시점에서, Union-find에 있는 간선들은 루트에서 현재 노드로 가는 경로 상에 있는 간선들의 합집합과 동일.

- 특정 노드를 떠나기 전 간선을 제거하지 않은 시점에서, Union-find에 있는 간선들은 루트에서 현재 노드로 가는 경로 상에 있는 간선들의 합집합과 동일하며, 이 간선들이 추가된 순서는 루트에서 현재 노드로 내려가는 방향임 (즉 마지막으로 진행한 연산을 undo하면 현재 노드에서 추가한 간선들이 undo됨.)

이제 각 시간에 대한 그래프에 대한 Union-find가 있으니, 쿼리는 리프 노드에서 응답하면 됩니다.

서브태스크 4

서브태스크 3과 동일하게, 간선 갱신을 "시간 구간 $[l, r]$ 에 간선 $(u, u + 1)$ 추가"의 형태로 변환합니다. 가능한 간선의 후보가 $N - 1$ 개이기 때문에, 위 표현을 뒤집어서 간선 추가 대신 "시간 구간 $[l, r]$ 에 u 와 $u + 1$ 사이에 간선이 없음"이라는 형태로 그래프의 상태를 표현할 수도 있습니다. 이렇게 $[l, r]$ 시간 구간에 u 와 $u + 1$ 이 연결되지 않았다는 정보를 튜플 (u, l, r) 의 형태로 표현합니다.

쿼리 (v, l', r') 이 주어졌을 때, 해당 쿼리에 대해 도달 가능한 정점은 구간을 이룹니다. 도달 가능한 정점 중 인덱스 최소를 x 라고 합니다. 각 튜플 (u, l, r) 를 2차원 평면 상에서 (u, l) 과 (u, r) 을 잇는 y 축에 평행한 선분으로 생각해 봅시다. $u < v$ 인 모든 선분 중 $[l', r']$ y 축 구간을 지나는 선분이 있다면, 이러한 선분 중 최대 $u + 1$ 이 우리가 구하는 x 입니다. 그러한 선분이 없을 경우 $x = 0$ 입니다.

이를 계산하는 방법은 다음과 같습니다:

- $u < v, l \in [l', r']$ 인 (u, l, r) 중 최대 u 구하기는, l 에 대해서 최댓값 세그먼트를 만들고 u 가 증가하는 순서대로 스위핑을 하면서 구할 수 있습니다.
- $u < v, r \in [l', r']$ 인 (u, l, r) 중 최대 u 구하기는, r 에 대해서 최댓값 세그먼트를 만들고 u 가 증가하는 순서대로 스위핑을 하면서 구할 수 있습니다.
- 위 경우는 모두 $[l', r']$ y 축 구간을 지나는 올바른 선분들만 고려했지만, 만약에 선분이 $l < l' \leq r' < r$ 을 만족한다면 해당 경우는 고려되지 않습니다. 이 경우는 선분을 y 축이 증가하는 순서대로 스위핑한 후, 각 쿼리에 대해서 (v, l') 에서 왼쪽으로 갔을 때 처음 닿는 선분을 구하는 식으로 계산할 수 있습니다. 정확하게 조건만 만족하는 선분이 고려되지는 않지만, 저 조건을 만족하는 선분은 전부 고려되며 그 외에 고려된 선분들도 전부 가능한 후보라서 괜찮습니다.

최대 v 는 대칭적으로 해결하면 됩니다. 전체 문제가 $O((N + S) \log(N + S) + Q \log Q + Q \log N)$ 시간에 해결됩니다.

만점 풀이

$s(u)$ 를, 정점 u 가 속한 연결 컴포넌트의 번호를 시간 순으로 나타낸 길이 $T + 1$ 의 문자열이라고 합니다. 또한, $s(u)[l \dots r]$ 을 이 문자열의 $l, l + 1, \dots, r$ 번째 문자로 이루어진 부분문자열이라고 합니다 ($0 \leq l \leq r \leq T$). 서브태스크 2에서는 $s(u)$ 를 직접 구한 후 적절한 전처리로 두 문자열 구간의 동일성을 비교했습니다. 만점 풀이에서도 유사한 방식의 풀이를 사용합니다.

서브태스크 3과 동일한 식으로 세그먼트 트리를 구성합니다. 세그먼트 트리의 노드 p 에 대해, 정점 v 가 활성화 되었다는 것은, p 와 p 의 서브트리에서 v 를 끝점으로 하는 간선이 존재한다는 것을 뜻합니다. 각 간선은 $O(\log T)$ 개의 새로운 활성화된 정점을 도입하니, 총 $O(S \log T)$ 개의 활성화된 정점이 세그먼트 트리에 존재합니다.

세그먼트 트리의 노드 p 에 대해서, 다음을 정의합니다:

- $ord(p, v)$: 노드 p 가 시간 구간 $[l, r]$ 을 대표한다고 하자. p 에 있는 모든 활성화된 정점들에 대해서, $s(v)[l \dots r]$ 를 사전순으로 비교했을 때 v 는 사전 순으로 몇 번째인가? (두 문자열이 같을 경우, 같은 번호를 배정)
- $rev(p, i)$: $ord(p, v) = i$ 인 정점은 무엇인가?
- $lcp(p, i)$: $rev(p, i)$ 와 $rev(p, i - 1)$ 이 시간 $[l, x]$ 에 연결되어 있게끔 하는 최대 x 는 무엇인가?
- $ord^R(p, v)$: x^R 을 x 를 뒤집은 문자열로 정의하자. $(s(v)[l \dots r])^R$ 를 사전순으로 비교했을 때 v 는 사전 순으로 몇 번째인가? (두 문자열이 같을 경우, 아무 순서대로 배정)
- $rev^R(p, i)$: $ord^R(p, v) = i$ 인 정점은 무엇인가?
- $lcp^R(p, i)$: $rev^R(p, i)$ 와 $rev^R(p, i - 1)$ 이 시간 $[l, x]$ 에 연결되어 있게끔 하는 최대 x 는 무엇인가?

이 정보를 저장하기 위해서는, p 에서 활성화된 정점에 비례하는 공간이 필요합니다. 고로 노드들의 공간 복잡도 합은 $O(S \log T)$ 입니다.

이제 위 정보들을 세그먼트 트리를 구성하면서 top-down으로 계산합니다. $m = (l + r)/2$ 라고 하고, v 가 Union-find의 루트라고 합시다 (아닌 경우는 $v = find(v)$ 로 맞춰 줍시다). $s(v)[l \dots r]$ 은, 다음 두 문자열을 합친 것과 동일합니다:

- 만약 v 가 왼쪽 자식에서 활성화된 노드라면, $s(v)[l \dots m]$. 아닐 경우 v 를 $m - l + 1$ 번 반복.
- 만약 v 가 오른쪽 자식에서 활성화된 노드라면, $s(v)[m + 1 \dots r]$. 아닐 경우 v 를 $r - m$ 번 반복.

$s(v)[l \dots r]$ 을 정렬하는 것은 이 두 쌍을 순서대로 정렬하는 것과 동일합니다. 왼쪽 자식에서 $s(v)$ 의 순서와 오른쪽 자식에서 $s(v)$ 의 순서가 이미 구해져 있으니, $ord(p, v)$ 는 $(ord(2p, v), ord(2p + 1, v))$ 를 구해서 사전순으로 정렬한 후 좌표압축하는 식으로 구할 수 있습니다. 만약 v 가 활성화된 노드가 아니라면, $ord(*, v)$ 를 $v + n$ 으로 대체하는 식으로 처리할 수 있습니다. 즉, 위와 같이 정의된 두 쌍을 정렬하는 식으로 ord, rev 를 $O(S \log T \log N)$ 시간에 계산할 수 있습니다. 위 자료들을 통해서 임의의 서브트리에서 두 노드가 같은 문자열을 가지는지를 $O(\log N)$ 시간에 판단할 수 있으니, 세그먼트 트리를 이분탐색 하는 식으로 $lcp(p, i)$ 역시 동일한 복잡도에 계산할 수 있습니다. $(\dots)^R$ 에 대한 값은 뒤집어서 동일하게 계산하면 됩니다.

이제 쿼리를 처리하는 부분을 설명합니다. 쿼리는 세그먼트 트리를 타고 내려가면서 처리해 줍니다 (구성하는 것과 같은 분할 정복에서 해결하는 것이 효율적입니다). 쿼리 구간 $[l', r']$ 을 포함하는 가장 깊은 노드를 p 라고 합시다. p 에서 v 가 활성화되어 있지 않다면, p 로 내려가면서 활성화 되지 않게 된 가장 첫 지점을 통해서 답을 쉽게 계산할 수 있습니다.

p 에서 v 가 활성화되어 있음을 가정합니다. $s(*)[l \dots m]^R, s(*)[m + 1, r]$ 을 기준으로 모든 활성화된 노드들이 정렬되어 있으니, $[l', m]$ 구간에서 문자열이 일치하는 v 는 $ord^R(2p, v)$ 값이 특정 구간에 있습니다. 이 구간은 $lcp^R(2p)$ 값을 사용하여 v 가 있는 지점 양 옆으로 세그먼트 트리에 이분탐색을 하여 계산할 수 있습니다. 비슷하게, $[m + 1, r']$ 구간에서 문자열이 일치하는 v 는 $ord(2p + 1, v)$ 값이 특정 구간에 있습니다. 고로 이 노드에서 우리가 구하는 것은, $ord^R(2p)$ 값이 특정 구간에 있고 $ord(2p + 1)$ 값이 특정 구간에 있는 모든 정점 v 에 대해서 Union-find 컴포넌트의 크기 합을 구하는 것으로 환원됩니다. 다시 말해, 현재 노드에서 활성화된 모든 정점들을 2차원 평면에 올렸을 때, 각 쿼리는 2차원 직사각형 안에 들어간 정점의 가중치 합을 묻는 문제가 됩니다. 이는 스윕핑을 사용하여 쉽게 해결할 수 있습니다.

최종 시간 복잡도는 $O(S \log T \log N + Q \log N)$ 입니다.

격자 트리 - 풀이

작성자: 장근영

부분문제 1

$N \leq 7$ 의 모든 경우에 대해 트리를 그려보면 된다. $N = 7$ 인 완전이진트리를 제외하고 모두 부분문제 2의 경우에 속한다. 완전이진트리에서 i 번 정점의 부모는 $\lfloor \frac{i-1}{2} \rfloor$ 번이라고 하자.

- $c_1 = c_2 = 1$ 인 경우에는 $\max(1 + c_3, 1 + c_4, 2 + c_5, 2 + c_6)$ 와 $\max(2 + c_3, 2 + c_4, 1 + c_5, 1 + c_6)$ 중 작은 값이 답이다. 0번 정점과 자식을 잇는 경로 중 하나의 길이를 2 이상이어야 하기 때문이다.
- $c_1 > 1$ 또는 $c_2 > 1$ 인 경우에는 $\max(c_1 + c_3, c_1 + c_4, c_2 + c_5, c_2 + c_6)$ 가 답이다.

부분문제 2

자식이 0개인 정점 v 에 대해 루트에서 v 까지의 경로에 포함된 간선에 대응하는 c_v 의 합을 d_v 로 정의하자. D 를 $\max(d_v)$ 로 정의하자.

트리의 격자 깊이가 D 보다 크거나 같음은 자명하다. 이제 D 의 격자 깊이를 가지는 트리를 그릴 수 있음을 귀납법으로 보이자. $N = 3$ 인 경우에는 쉽게 보일 수 있으니, $N > 3$ 이라고 가정하자.

편의상 0번 루트 정점의 왼쪽/오른쪽 자식을 1, 2번 정점이라고 하자. 이때 2번 정점은 자식이 없는 정점이다. $m_1 = (c_1, 0), m_2 = (0, D)$ 에 그리자. 귀납법에 의해 1번 정점의 서브트리를 격자 깊이 $D - c_1$ 에 그릴 수 있다. 이를 x 방향으로 c_1 만큼 평행이동하자. 그러면 전체 트리를 그릴 수 있고, 이 트리의 격자 깊이가 D 이다.

부분문제 3

부분문제 3에서는 정점 v 와 그 부모 정점을 잇는 경로의 길이가 c_v 로 고정된다. 주어진 최소 길이 조건에 맞춰 트리를 그릴 수 있는지 판별하는 문제로 환원된다.

문제를 단순화하기 위해, 길이 k 의 경로를 1인 단위 경로들로 분할하고, 그 사이에 $k - 1$ 개의 가상 정점을 추가하자. 이제 문제는 다음과 같이 바뀐다.

- 자식의 개수가 2개 이하이고 모든 간선의 길이가 1인 트리가 주어졌을 때, 이 트리를 격자 조건에 맞춰 그릴 수 있는가?

이 변환 과정을 거치면 루트로부터 각 정점까지의 깊이는 유일하게 결정된다. 이때 트리를 올바르게 그릴 수 있기 위한 필요충분조건은 다음과 같다.

- 각 정점 v 와 모든 자연수 k 에 대해, v 를 루트로 하는 서브트리에서 v 와의 깊이 차이가 k 인 자손 정점의 개수는 $k + 1$ 개 이하이다.

이에 대한 증명은 풀이의 가장 마지막 부분에 있다. 필요충분조건은 $O(n^2)$ 에 검증할 수 있다.

부분문제 4

각 서브트리가 점유해야 하는 최소한의 기하학적 영역을 너비 수열로 정의하고, 이를 리프에서부터 귀납적으로 병합하는 $O(N^2)$ 동적 계획법 풀이를 제시한다.

정의 및 성질

정점 (x, y) 의 깊이를 $x + y$ 로 정의한다.

어떤 정점 v 의 서브트리를 격자 트리에 그렸을 때 v 에서 v 의 가장 왼쪽/오른쪽 리프로 이어지는 경로로 이루어진 영역을 껍질이라고 하자.

임의의 서브트리 T 에 대하여, 너비 수열 $L(T) = [l_0, l_1, \dots, l_k]$ 는 해당 서브트리를 격자상에 구현하기 위해 필요한 높이별 최소 너비를 나타낸다.

- 격자점의 높이를 (리프 노드의 깊이) - (정점의 깊이)로 정의하자. 리프 정점에 대응하는 격자점의 높이는 0이다.
- l_i 는 높이 i 에서 서브트리의 노드들이 점유하는 y 좌표의 구간 길이의 하한값이다.
- 서브트리의 루트는 단일 정점이므로 너비가 0이며, 수열은 항상 $l_k = 0$ 으로 종료된다.

너비 수열 $L(T)$ 는 서브트리 T 를 내부에 포함할 수 있는 최소한의 껍질의 각 층별 너비를 정의한다. 루트 정점에서 리프 정점을 향하는 경로의 이동 제약에 의해, 껍질의 경계는 급격한 기울기 변화를 가질 수 없다. 이를 좋은 수열 조건이라 한다.

- 유효한 너비 수열 l 은 모든 $i > 0$ 에 대해 $l_i \geq l_{i-1} - 1$ 을 만족해야 한다.

이는 높이가 1 증가할 때, 껍질의 너비가 최대 1만큼만 감소할 수 있음을 의미한다. 기하학적으로는 껍질의 외벽이 내부로 45° 보다 급격하게 꺾여 들어올 수 없음을 시사한다.

너비 수열은 아래와 같은 조건을 만족한다. 이를 다음 문단에서 너비 수열을 구하는 과정에서 증명한다.

- 서브트리 T 에 대해 왼쪽 리프를 향하는 경로와 오른쪽 리프를 향하는 경로가 모든 높이 i 에서 너비(y 값 차이)가 $L(T)_i$ 인 껍질이 존재한다면, 껍질 내부에 T 에 대응하는 격자 트리를 그릴 수 있다.

너비 수열 구하기

다이나믹 프로그래밍을 통해 각 정점별로 서브트리의 너비 수열을 구해보자.

자식이 없는 리프 노드는 높이가 0이고 너비가 0인 단일 점이므로, 다음과 같이 초기화한다.

$$L(\text{leaf}) = [0]$$

어떤 노드 u 가 두 자식 v_L, v_R 을 가지며, 각각의 너비 수열을 P, Q 라고 하자. u 와 v_L / v_R 사이의 최소 경로 길이를 c_L / c_R 이라고 하면, P 와 Q 의 뒤에 각각 c_L / c_R 개의 0을 붙여 새로운 너비 수열을 만들자.

모든 리프는 동일한 격자 깊이를 가지므로, u 의 서브트리에서 높이 i 의 너비는 $P_i + Q_i + 1$ 이상임을 알 수 있다.

결합된 임시 수열 S 는 다음과 같이 정의하자:

$$S_i = P_i + Q_i + 1$$

(단, P, Q 의 길이가 다를 경우, 정의되지 않은 인덱스의 값은 0으로 간주한다.)

단순 병합된 수열 S 는 좋은 수열 조건을 위배할 수 있다. 따라서 격자 경로의 연속성을 보장하기 위해, 아래층의 너비를 기반으로 위층의 너비를 강제로 확보하는 연산을 수행한다.

보정된 수열 R 은 다음과 같이 정의된다. R 은 u 의 너비 수열 $L(u)$ 가 된다.

$$R_0 = S_0$$

$$R_i = \max(S_i, R_{i-1} - 1) \quad (\text{for } i > 0)$$

새로 만든 수열 $L(u)$ 가 위에서 언급한 껍질과 너비 수열의 대응 조건을 만족함을 보는 과정에 대해 간략하게 설명하자.

높이 i 에서의 너비가 $L(u)_i$ 인 껍질 C 내부에는, 왼쪽 경계를 따라 높이 i 에서 너비가 P_i 인 껍질 C_L 과 오른쪽 경계를 따라 높이 i 에서 너비가 Q_i 인 껍질 C_R 를 겹치지 않게 만들 수 있음을 어렵지 않게 증명할 수 있다. C_L 과 C_R 안에 각각 v_L 과 v_R 의 서브트리에 대응하는 격자 트리를 그린 이후 두 격자 트리를 병합해주면 C 내부에 u 의 서브트리에 대응하는 격자 트리를 그릴 수 있음을 증명할 수 있다.

너비수열의 길이는 리프 개수와 리프의 최대 깊이(루트에서 리프까지의 경로에서 c_v 의 합)의 합 이하임을 보일 수 있다. 리프의 최대 깊이를 X 라고 할 때 $O(N^2 + NX)$ 의 시간복잡도로 문제를 해결할 수 있다.

전체 풀이

각 정점의 너비 수열 L 에 대해 $L_i \neq L_{i+1}$ 인 i 의 집합 S_L 을 관리하자.

두 너비 수열 P 과 Q 를 합치는 경우, S_Q 의 원소 x 에 대해 S_P 에 속하지 않는 가장 작은 x 이상의 값을 S_P 에 추가하는 시행을 반복하면 된다. S_L 을 `std::set`으로 관리하자. small to large 기법을 사용해 $O(N \log^2 N)$ 에 전체 문제를 해결할 수 있다.

부분문제 3의 필요충분조건 증명

해당 조건이 필요조건인 것은 자명하다. 우리는 앞서 도출한 조건이 트리를 그리기 위한 충분조건임을 구성적 증명을 통해 보인다. 증명은 트리의 높이(리프부터 루트까지의 경로 길이)에 대한 수학적 귀납법을 사용한다.

1. 정의 및 세팅

- 경로 분할
입력으로 주어진 각 정점 v 와 그 부모를 잇는 경로의 길이가 c_v 일 때, 이를 길이 1인 단위 경로 c_v 개로 분할하고 그 사이에 $c_v - 1$ 개의 가상 정점을 추가한다. 이제 인접한 정점 사이의 모든 경로의 길이는 1로 통일된다. 이 과정에서 가상 정점들도 편의상 자식이 1개인 정점으로 취급하며, 원래 v 가 부모의 왼쪽 자식이었다면 분할된 경로의 가장 위쪽(부모와 연결되는) 가상 정점이 왼쪽 자식의 성질을 갖는다고 간주한다.
- 초기 리프 배치
왼쪽에서 오른쪽으로 정렬된 $M = \frac{N+1}{2}$ 개의 리프 정점들을 좌표 평면 상의 대각선 $x + y = K$ 위에 배치한다. 왼쪽에서 i 번째 리프 정점 l_i 의 좌표는 $(K - (i - 1), i - 1)$ 로 설정한다. 따라서 리프 정점들의 y 좌표는 $0, 1, \dots, M - 1$ 로 고정된다.
- 밀집 상태
정점 x 에 대해 x 의 서브트리에 있는 리프 정점들의 y 좌표가 연속하다면 x 가 밀집 상태에 있다고 한다. 정의에 의해 모든 리프는 밀집 상태이다.
- L / R 방향

부모를 L / R 방향으로 결정한다는 것은, 현재 정점의 위치에 $(-1, 0) / (0, -1)$ 을 더해 부모 정점의 위치를 정한다는 것이다.

2. 귀납법을 통한 증명

각 정점의 자식 개수가 2개 이하인 트리가 주어지고, 리프 정점들의 좌표가 고정되어 있다고 하자. 이때, 주어진 트리와 리프의 배치가 다음 두 조건을 모두 만족한다면, 트리를 격자 평면 위에 올바르게 그릴 수 있다.

- 너비 조건: 임의의 정점 x 에 대하여, x 의 왼쪽 자식을 타고 내려가서 도달하는 리프 u 와, x 의 오른쪽 자식을 타고 내려가서 도달하는 리프 v 에 대해 다음 부등식이 성립한다.

$$y_v - y_u \leq \text{depth_diff}(x)$$

(여기서 $\text{depth_diff}(x)$ 는 정점 x 와 리프 정점 사이의 깊이 차이를 의미하며, y_u, y_v 는 리프 정점의 y 좌표를 의미한다.)

- 밀집도 조건: 임의의 정점 x 와 자연수 k 에 대하여, x 의 자손 중 x 와의 깊이 차이가 k 인 정점의 개수는 $k + 1$ 개 이하이다.

트리의 깊이에 대한 귀납법을 사용하자. 깊이가 $h + 1$ 인 트리에서 위 두 조건이 성립할 때, 현재 리프들의 부모 정점을 적절히 배치한 이후 리프들을 제거해서 깊이가 h 인 새로운 트리를 만든다. 새로 만든 트리가 위 귀납 가정을 만족함을 보임으로써 증명을 완성할 것이다.

3. 부모 정점 좌표 결정 전략

깊이가 $h + 1$ 인 트리의 리프 정점들의 위치가 고정되어 있을 때, 각 리프의 바로 위 부모 정점들의 위치를 결정한다. 현재 트리의 리프들을 모두 제거하면 깊이가 h 인 트리가 되며, 원래 리프들의 부모 정점들이 이 새로운 트리의 리프(이하 새로운 리프)가 된다.

각 정점 x 에 대해, x 의 서브트리에 속한 현재 리프들이 x 쪽(부모 방향)으로 이동할 방향은 다음 알고리즘에 따라 결정한다.

- x 가 밀집 상태인 경우: x 가 어떤 정점의 왼쪽 자식이라면 모든 리프의 부모를 L 방향으로, 오른쪽 자식인 경우 R 방향으로 결정한다.
- x 가 밀집 상태가 아닌 경우: 양쪽 자식에 대해 재귀적으로 부모 방향을 결정한다.

단, 전체 트리의 루트가 밀집 상태라면 모든 리프의 부모 방향을 L 방향으로 결정한다.

4. 유효성 증명

새로운 리프들의 위치를 결정하면서, 부모가 같은 두 리프 정점이 같은 부모 위치를 결정했는지, 부모가 다른 두 리프 정점이 다른 부모 위치를 결정했는지를 확인해야 한다. 또한 귀납 조건을 잘 유지했는지 보여야 한다. 밀집도 조건은 트리의 구조적인 성질이므로 너비 조건이 잘 성립하는지만 확인하면 된다.

1) 너비 조건 확인

- 정점 x 가 밀집 상태가 아닌 경우
 x 의 서브트리 내부 가장 왼쪽 자식은 L 방향을 선택했고, 가장 오른쪽 자식은 R 방향을 선택했다. 두 리프들 사이의 y 좌표 차이(너비)는 1 감소하고, x 까지의 높이 차이 또한 1 감소한다. 부등식의 양변이 같이 1씩 감소하므로 조건은 유지된다.
- 정점 x 가 밀집 상태인 경우
너비 조건이 유지됨은 원래 트리의 밀집도 조건에 의해 보장된다.

2) 같은 부모 정점을 가지는 두 리프의 부모 위치

- 같은 부모 p 를 공유하는 두 리프 정점 u (왼쪽 자식), v (오른쪽 자식)가 있다고 하자. 이들은 이번 단계에서 정확히 하나의 정점 p 로 병합되어야 한다.
- u 와 v 가 같은 부모를 가지므로, u 와 v 를 포함하는 서브트리는 밀집 상태가 아님을 알 수 있다. 따라서 u 는 L 방향으로, v 는 R 방향으로 부모 위치를 결정한다.
- 너비 조건에 의해 u 와 v 의 y 좌표 차이는 부모 p 와의 깊이 차이인 1보다 작거나 같아야 한다. 서로 다른 두 리프는 겹칠 수 없으므로, u 와 v 사이의 간격은 정확히 1이다.
- 따라서 u 와 v 의 부모 위치는 일치한다

3) 다른 부모 정점을 가지는 두 리프의 부모 위치

두 리프 정점의 부모가 다르지만 결정된 부모 위치가 같다고 가정하자. 두 리프의 y 좌표 차이는 1이다.

- 두 정점의 조상 중 자식을 2개 가지는 가장 가까운 조상이 x 로 같은 경우:
 x 는 밀집 상태에 있으므로 두 정점은 같은 방향으로 부모를 선택하게 되고, 결정된 두 부모의 위치는 다르기 때문에 모순이다.
- 두 정점의 조상 중 자식을 2개 가지는 가장 가까운 조상이 다른 경우: 두 리프를 모두 서브트리 내부에 가지는 가장 가까운 정점을 x 라고 하자. 한 리프 u 는 x 의 왼쪽 자식 l 의 가장 오른쪽 리프이고, 다른 리프 v 는 x 의 오른쪽 자식 r 의 가장 왼쪽 리프이다. x 가 밀집 상태인 경우에는 같은 논리로 모순이다. l 과 r 이 동시에 밀집상태인 경우는 x 가 밀집 상태인 것과 동일하기 때문에 일어날 수 없다. 따라서 u 가 R 방향으로 부모를 결정하거나 v 가 L 방향으로 부모를 결정하고, 두 리프의 부모 위치는 다르게 결정된다.

따라서 새로 구성된 높이 h 인 트리와 새로운 리프들은 귀납 가정을 모두 만족한다. 귀납 가설에 의해 전체 트리를 그릴 수 있다. ■

멋진 구간 2 - 풀이

작성자: 박영우

부분문제 1

$[A[l], \dots, A[r]]$ 에 연산을 적용하여 $[B[l], \dots, B[r]]$ 로 만들 수 있다고 하자. 이 과정에서 연속하게 시행된 어떤 두 연산에 대하여 처음 시행된 연산에서 증가시킨 값보다 나중에 시행된 연산에서 증가시킨 값이 더 작다면, 두 연산의 순서를 바꿔서 시행해도 된다. 따라서 $[l, r]$ 이 좋은 구간인 것은 $A[i] < B[i]$ 인 인덱스 i 중 $A[i]$ 가 가장 작은 인덱스부터 $A[i]$ 를 증가시키는 순서대로 연산을 시행하여 $[B[l], \dots, B[r]]$ 로 만들 수 있는 것과 필요충분조건이다.

이를 쿼리마다 $O(N^2 \max\{B_i\})$ 에 확인해주면 1번 부분문제를 해결할 수 있다.

부분문제 2

배열 A 가 1로만 구성되어 있을 경우, $[l, r]$ 이 좋은 구간일 필요충분조건은 다음과 같다.

- $[B[l], \dots, B[r]]$ 에 1 이상 $\max_{l \leq i \leq r} \{B[i]\}$ 이하의 원소가 적어도 한 번씩 등장한다.

다음과 같이 증명할 수 있다.

$B_{max} = \max_{l \leq i \leq r} \{B[i]\}$ 로 정의하자.

- $\{B[l], \dots, B[r]\} = \{1, \dots, B_{max}\}$ 라 가정하면, 부분문제 1에서 사용한 알고리즘이 항상 성공함을 알 수 있다. 따라서 $[l, r]$ 은 좋은 구간이다.
- 반대로, $\{B[l], \dots, B[r]\} \neq \{1, \dots, B_{max}\}$ 라 가정하자. 이는 $c \in \{1, \dots, B_{max}\}, c \notin \{B[l], \dots, B[r]\}$ 인 정수 c 가 존재함을 의미한다. $B_{max} \in \{B[l], \dots, B[r]\}$ 이므로 $c < B_{max}$ 이다. 만약 $[l, r]$ 이 좋은 구간이라면, $c < B_{max}$ 이므로 연산이 시행되는 동안 c 는 적어도 한 번의 시점에서 배열에 등장한다. 그러나 그 시점 이후 적어도 하나의 인덱스는 c 이고, 배열의 최종 상태에서는 c 가 없어야 하기 때문에 모순이다.

위 필요충분조건을 쿼리당 $O(N)$ 또는 $O(N \lg N)$ 정도에 확인하면 해당 부분문제를 풀 수 있다.

부분문제 3

부분문제 2의 필요충분조건을 쿼리당 $O(\lg N)$ 의 시간복잡도에 확인하면 된다. $\{B[l], \dots, B[r]\} = \{1, \dots, B_{max}\}$ 일 필요충분조건은 두 집합의 크기가 같다는 것이고, $|\{B[l], \dots, B[r]\}| = B_{max}$ 와 동치이다. $B[l], \dots, B[r]$ 에서의 서로 다른 수의 개수를 세는 것은 세그먼트 트리를 통해 총 Q 개의 쿼리에 대해 $O((N + Q) \lg N)$ 시간복잡도에 구할 수 있고, $\max_{l \leq i \leq r} \{B[i]\}$ 도 세그먼트 트리를 통해 같은 시간 복잡도에 구할 수 있다. 따라서 전체 문제를 $O((N + Q) \lg N)$ 에 풀 수 있다.

부분문제 4

$l \leq r$ 인 두 정수 l, r 에 대해 $[l, r)$ 을 다음과 같이 정의하자.

- $l = r$ 이면 $[l, r) = \emptyset$
- $l < r$ 이면 $[l, r) = \{l, l+1, \dots, r-1\}$

다음과 같은 관찰을 할 수 있다.

$[l, r]$ 이 좋은 구간일 필요충분조건은 다음 조건을 만족하는 것이다.

- 모든 $l \leq i \leq r$ 인 i 에 대하여 $[A[i], B[i]) \subseteq \{B[l], \dots, B[r]\}$ 를 만족한다. 즉, $\bigcup_{l \leq i \leq r} [A[i], B[i]) \subseteq \{B[l], \dots, B[r]\}$ 이다.

다음과 같이 증명할 수 있다.

1. 조건을 만족하면 좋은 구간이다. $A[i] < B[i]$ 이며 $A[i]$ 가 가장 작은 인덱스 i 를 고른다. 그러한 인덱스가 없다면 이미 조건을 충족한다.
어떤 $B[j]$ 가 있어 $A[i] = B[j]$ 이다. 만약 $i = j$ 라면 $A[i] = B[i]$ 이므로 $i \neq j$ 이다.
만약 $A[j] < B[j]$ 일 경우 $A[j] < A[i]$ 이므로 i 의 정의에 모순이다.
따라서 $A[j] = B[j]$ 이고, (i, j) 에 대해 연산을 시행하면 된다.
2. 좋은 구간은 항상 위 조건을 만족한다.
연산은 배열의 값을 증가시키므로 첫 번째 조건은 항상 충족한다. 만약 어떤 배열에 x 라는 값이 있었다면 이 상태에서 연산을 몇 번을 시행하더라도 배열에 여전히 x 는 존재한다.
만약 조건을 만족하지 않는 인덱스 i 가 있고, 어떤 $c \in [A[i], B[i])$ 가 있어 $c \neq B[l], \dots, B[r]$ 이라 하자.
어떤 시점에서 i 번 인덱스의 값은 c 에서 $c+1$ 로 증가했고, 이 시점에서 배열에는 값이 c 인 인덱스가 존재했다. 따라서 배열의 최종 상태 $[B[l], \dots, B[r]]$ 에도 c 가 존재하며, 모순이다.

이 필요충분조건을 쿼리당 $O(N)$ 또는 $O(N \lg N)$ 시간에 확인하면 해당 부분문제를 해결할 수 있다.

부분문제 5

다양한 풀이가 존재한다. $[l, r]$ 에 $A[i] = A[j], B[i] = B[j]$ 인 두 인덱스 i, j 가 있다면 두 인덱스 중 하나만 고려해도 된다.

$B[i] \leq 2$ 일 경우 $(A[i], B[i])$ 로 가능한 순서쌍은 $(1, 1), (1, 2), (2, 2)$ 로 총 세가지로, 구간에 각각에 해당하는 인덱스가 있는지 확인한 후 총 2^3 가지 경우에 대해 문제를 해결해주면 된다.

부분문제 6

입력으로 주어진 배열 $[B[1], B[2], \dots, B[N]]$ 의 최댓값을 B 라 하자.

부분문제 4의 필요충분조건을 쿼리당 $O(B)$ 에 확인하면 된다.

모든 $1 \leq i \leq N, 1 \leq j \leq B$ 인 인덱스 i, j 에 대해 다음 두 값을 구한다.

- 1 이상 i 이하인 모든 인덱스 k 중 $j \in [A[k], B[k])$ 인 인덱스의 개수
- 1 이상 i 이하인 모든 인덱스 k 중 $j = B[k]$ 인 인덱스의 개수 다음 두 값은 전체 $O(NB)$ 에 구할 수 있다.

따라서 각 질문 $[l, r]$ 에 대하여 1 이상 B 이하의 임의의 원소 c 에 대해 $c \in \bigcup_{l \leq i \leq r} [A[i], B[i]]$ 인지, $c \in \{B[l], \dots, B[r]\}$ 인지 여부를 알 수 있다.

따라서 각 쿼리당 $O(B)$ 의 시간에 문제를 해결할 수 있고, 총 $O((N + Q)B)$ 시간복잡도에 문제를 해결할 수 있다.

부분문제 7

l 을 고정하고, 구간의 왼쪽 끝점이 l 인 질문들만 고려하자.

1 이상 $B (= \max_{1 \leq i \leq N} \{B[i]\})$ 이하인 모든 정수 c 에 대해 $c \in [A_i, B_i]$ 이며 $l \leq i$ 인 최소 인덱스 i 를 X_c , $c = B_i$ 이며 $l \leq i$ 인 최소 인덱스를 Y_c 라 하자.

$[l, r]$ 이 좋은 구간이 아닐 필요충분조건은 어떤 c 가 존재하여 $c \notin \{B[l], \dots, B[r]\}$ 이면서 $c \in \bigcup_{l \leq i \leq r} [A[i], B[i]]$ 인 것이다. 즉, 어떤 c 가 존재하여 $X_c \leq r < Y_c$ 를 만족한다면 $[l, r]$ 은 좋은 구간이 아니다.

각각의 r 에 대해 $X_c \leq r < Y_c$ 를 만족하는 서로 다른 정수 c 의 개수를 세그먼트 트리를 통해 관리한다. 이 세그먼트 트리의 i 번째 인덱스를 $seg[i]$ 라 하자. 즉, $X_c < Y_c$ 인 c 는 $i \in [X_c, Y_c]$ 인 모든 정수 i 에 대해 $seg[i]$ 에 1 만큼 기여한다.

이를 위해, 다음 집합들을 관리한다.

- $X_c < Y_c$ 인 c 들의 집합 S
- $X_c \geq Y_c$ 인 c 들의 집합 T

고정된 왼쪽 끝점 l 을 $l - 1$ 로 줄일 때 $c \in [A[l - 1], B[l - 1]]$ 인 모든 X_c 와 $Y_{B[l - 1]}$ 은 $l - 1$ 로 감소한다.

- 업데이트되는 X_c 들의 경우, 변경되기 전 X_c 값이 같은 구간들로 분할하고 같은 구간에 포함되는 인덱스들에 대해 세그먼트 트리를 한번에 업데이트한다. 배열 X 를 `std::set`, 세그먼트 트리 등의 자료구조를 통해 관리하면 한 구간 당 $O(\lg B)$ 의 시간에 찾을 수 있다.
 - $c \in [s, e]$ 인 모든 c 에 대해 X_c 의 값이 p 에서 $l - 1$ 로 업데이트된다 하자.
 - 업데이트 전 $[s, e] \cap S$ 에 있는 원소 c 들은 업데이트 이후에도 S 에 있어야 하며, $seg[X_c], \dots, seg[Y_c - 1]$ 에 1씩 기여하고 있다. 따라서 X_c 의 값을 $l - 1$ 로 바꿔주기 위해 $seg[l - 1], \dots, seg[p - 1]$ 에 $|[s, e] \cap S|$ 를 더해줘야 한다. $|[s, e] \cap S|$ 는 S 를 세그먼트 트리, `bbst` 등의 자료 구조를 통해 관리하면 $O(\lg B)$ 시간에 구할 수 있다.
 - 업데이트 후 $[s, e] \cap T$ 에 있는 원소 c 들은 업데이트 이후에는 S 에 있게 된다. 따라서 $seg[l - 1], \dots, seg[Y_c - 1]$ 에 1이 더해줘야 한다.
- $Y_{B[l - 1]}$ 의 경우, $(X_{B[l - 1]}, Y_{B[l - 1]})$ 을 다시 계산하여 세그먼트 트리에 업데이트해주면 된다. 또한, 이 시점부터 $B[l - 1]$ 은 T 에 있게 된다.

위 과정에 따라 l 을 $N - 1$ 부터 0까지 줄여가며 세그먼트 트리를 업데이트해주고, 각각의 $[l, r]$ 쿼리에 대해 $seg[r]$ 이 0이라면 $[l, r]$ 이 좋은 구간이라 판정하는 방법으로 풀 수 있다. 이에 사용되는 시간복잡도는 $O(Q \lg B)$ 이다.

고정된 왼쪽 끝점을 l 에서 $l - 1$ 로 감소시킬 때 세그먼트 트리의 업데이트 과정에서 X 배열의 변화에 의해 세그먼트 트리를 업데이트하는 횟수는 다음 값의 합에 비례한다.

- 업데이트되는 서로 다른 구간 $[s, e]$ 의 수: $i \in (A[l - 1], B[l - 1]), X_{i-1} \neq X_i$ 인 인덱스 i 의 개수
- $|T \cap [A[l - 1], B[l - 1]]|$

업데이트 이후 $X_{A[l-1]}, \dots, X_{B[l-1]-1}$ 의 값은 모두 $l-1$ 로 변하므로, 전체 알고리즘이 시행되는 동안 첫 번째 값의 합은 $O(N)$ 이다.

$T \cap [A[l-1], B[l-1]]$ 에 속한 모든 원소 c 들은 업데이트 이후 T 에서 S 로 이동하고, 고정된 왼쪽 끝점을 한 번 옮기는 동안 최대 하나의 값($B[l-1]$)만 S 에서 T 로 이동하므로 전체 알고리즘이 시행되는 동안 두 번째 값의 합은 $O(N)$ 이다.

또한, 고정된 왼쪽 끝점을 l 에서 $l-1$ 로 감소시킬 때 세그먼트 트리의 업데이트 과정에서 Y 배열의 변화에 의해 세그먼트 트리를 업데이트하는 횟수는 1회이다.

따라서 전체 시간복잡도는 $O((N+Q) \lg B)$ 이다.

부분문제 8

$[A[i], B[i]] \not\subseteq \{B[0], \dots, B[N-1]\}$ 인 인덱스 i 들은 `std::set` 등의 자료구조를 통해 $O(N \lg N)$ 시간에 전부 구할 수 있다. 이러한 인덱스들을 전부 모은 집합을 I 라 하자, 만약 어떤 구간 $[l, r]$ 이 I 에 있는 인덱스 중 하나를 포함했다면 이 구간은 반드시 좋은 구간이 아니다.

이 사실을 이용하면 부분 문제 4의 필요충분조건을 다음과 같이 변형할 수 있다.

- $[l, r] \cap I = \emptyset$ 이고,
- $\bigcup_{l \leq i \leq r} [A[i], B[i]] \cap \{B[0], \dots, B[N-1]\} \subseteq \{B[l], \dots, B[r]\}$ 이다.

즉, I 의 원소를 포함하는 $[l, r]$ 의 구간을 전처리해 주면 A, B 배열을 좌표압축한 이후 부분 문제 7의 풀이를 그대로 적용할 수 있다.

전체 시간복잡도는 $O((N+Q) \lg N)$ 이다.